

线性注意力与 GDN

WPC

2026 年 5 月 27 日

目录

1 线性注意力 (Linear Attention)	2
1.1 从 Softmax 注意力到线性注意力	2
1.2 RNN 视角与递归更新	2
2 Gated Delta Networks (GDN)	3
2.1 GDN 的动机	3
2.2 Delta 更新规则	3
2.3 门控机制 (Gating)	3
2.4 GDN 的优势与未来展望	3
3 总结	4

1 线性注意力 (Linear Attention) 深度解析

1.1 Softmax 注意力的计算瓶颈

传统的 Softmax 注意力定义为：

$$O = \text{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right)V, \quad Q, K, V \in \mathbb{R}^{N \times d} \quad (1)$$

其中 $A = \text{softmax}(QK^\top/\sqrt{d})$ 是 $N \times N$ 的注意力矩阵。当序列长度 N 达到万级甚至十万级时，存储 A 所需的内存和计算 $A \times V$ 所需的计算量呈平方级增长，成为模型扩展的主要障碍。

1.2 核方法与结合律优化

线性注意力的核心是通过核函数 $\phi(\cdot)$ 将相似度计算从指数形式转化为向量内积形式。通用的线性注意力框架如下：

$$\text{Sim}(Q_i, K_j) = \phi(Q_i)\phi(K_j)^\top \quad (2)$$

此时，第 i 个位置的输出可以表示为：

$$O_i = \frac{\sum_{j=1}^N (\phi(Q_i)\phi(K_j)^\top)V_j}{\sum_{j=1}^N \phi(Q_i)\phi(K_j)^\top} = \frac{\phi(Q_i) \sum_{j=1}^N (\phi(K_j)^\top V_j)}{\phi(Q_i) \sum_{j=1}^N \phi(K_j)^\top} \quad (3)$$

通过先计算 K 和 V 的外积和（即隐藏状态 $S = \sum \phi(K_j)^\top V_j$ ），我们将复杂度降为 $O(N \cdot d^2)$ 。

1.2.1 核函数的选择 (Feature Maps)

为了保证注意力的非负性和稳定性，常用的核函数包括：

- **ELU 特征映射**： $\phi(x) = \text{elu}(x) + 1$ 。这是 Linear Transformer 论文中提出的经典方法，确保了输出全为正值。
- **ReLU 特征映射**：直接使用 $\text{relu}(x)$ 。简单有效，但在某些任务上可能导致稀疏性过强。
- **泰勒展开近似**：利用 $e^{x^\top y} \approx 1 + x^\top y + \frac{1}{2}(x^\top y)^2$ 。通过二阶项的近似，可以更逼近 Softmax 的行为，但计算量会有所增加。

1.3 因果掩码 (Causal Masking) 的线性实现

在自回归生成任务中，位置 i 只能看到位置 $j \leq i$ 的信息。对于 Softmax 注意力，这通过三角掩码实现。在线性注意力中，这转化为前缀和 (Prefix Sum) 的计算：

$$S_i = \sum_{j=1}^i \phi(K_j)^\top V_j = S_{i-1} + \phi(K_i)^\top V_i \quad (4)$$

$$z_i = \sum_{j=1}^i \phi(K_j)^\top = z_{i-1} + \phi(K_i)^\top \quad (5)$$

$$O_i = \frac{\phi(Q_i) S_i}{\phi(Q_i) z_i} \quad (6)$$

这种递归形式使得线性注意力在逻辑上等价于一个具有无限维状态的 RNN，但在训练时可以通过并行前缀扫描 (Parallel Prefix Scan) 实现与 Transformer 相当的训练速度。

2 Gated Delta Networks (GDN) 进阶探讨

2.1 从线性联想记忆到 Delta 规则

传统的线性注意力可以看作是一个简单的累加器： $S_t = S_{t-1} + K_t^\top V_t$ 。这种方式虽然简单，但存在记忆过载的问题——新的信息只是机械地叠加在旧信息之上。

GDN 引入了 **Delta 规则 (Delta Rule)**，其灵感来源于“快权重程序员” (Fast Weight Programmers) 和线性联想记忆的在线更新。Delta 规则的目标是让模型学会“更新”记忆而不是简单的“堆积”：

$$S_t = S_{t-1} + \alpha_t K_t^\top (V_t - K_t S_{t-1}) \quad (7)$$

其中：

- $K_t S_{t-1}$ ：这是根据当前的键 K_t 从旧记忆中提取出的预测值。
- $V_t - K_t S_{t-1}$ ：预测误差 (Delta)。
- α_t ：学习率 (通常由输入动态生成)。

通过减去预测值，模型能够自动遗忘旧记忆中与当前输入冲突的部分，从而为新信息腾出空间。

2.2 精细化门控机制 (Fine-grained Gating)

GDN 的核心提升在于其复杂的门控架构。典型的 GDN 单元包含以下组件：

2.2.1 1. 衰减门 (Decay Gate) f_t

用于控制长期记忆的保留程度。不同于传统 RNN 的标量门控，GDN 往往使用逐元素 (element-wise) 的衰减：

$$f_t = \text{sigmoid}(W_f x_t + b_f) \in (0, 1)^{d \times d} \quad (8)$$

2.2.2 2. 写入门 (Write Gate) w_t

控制当前 Delta 更新的强度：

$$w_t = \tanh(W_w x_t + b_w) \quad (9)$$

2.2.3 3. 完整的状态更新方程

结合上述组件，GDN 的隐藏状态 S_t (通常是一个 $d \times d$ 的矩阵) 更新如下：

$$S_t = f_t \odot S_{t-1} + w_t \odot (K_t^\top (V_t - \beta_t K_t S_{t-1})) \quad (10)$$

这里 β_t 是另一个动态生成的权重，用于调节 Delta 规则的干预程度。

2.3 硬件感知的块并行化 (Block Parallelization)

为了在 GPU 上高效训练，GDN 采用了类似于 FlashAttention 的块处理技术。虽然其逻辑是递归的，但在训练阶段，可以将序列划分为长度为 B 的块。

- **块内**：由于 Delta 规则的强耦合性，块内计算往往需要精细设计的 CUDA Kernel。
- **块间**：通过类似于 RNN 的状态传递，计算每个块的初始状态。

这种设计使得 GDN 在处理超长序列（如 128k 或更多）时，速度远超传统的 Transformer。

2.4 GDN vs. Mamba vs. Transformer

特性	Transformer	Mamba (SSM)	GDN
复杂度	$O(N^2)$	$O(N)$	$O(N)$
推理内存	$O(N)$ (KV Cache)	$O(1)$	$O(1)$
记忆机制	全局注意力	状态空间扫描	矩阵式联想记忆
更新规则	无（静态叠加）	线性微分方程	Delta 规则（误差修正）

表 2.1: 主流模型对比

GDN 通过将隐藏状态从向量扩展到矩阵，并在矩阵上应用 Delta 规则，实现了比 Mamba 更强的联想记忆能力，同时保持了线性的扩展性。